

**LAPORAN PRAKTIKUM ALGORITMA
DAN PEMROGRAMAN 2**

**MODUL 18
MESIN ABSTRAK**



Disusun oleh:
MANGGALA PATRA RADITYA
109082500179
S1IF-13-02

Asisten Praktikum
Muhammad Agha Zulfadhli
Renisa Assyifa Putri

PROGRAM STUDI S1 INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026
MODUL 18

1. Soal 1

Source code

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

type Domino struct {
    suit1 int
    suit2 int
    balak bool
}

type Dominoes struct {
    kartu []Domino
    sisa  int
}

func buatDomino() Dominoes {
    var d []Domino

    for i := 0; i <= 6; i++ {
```

```

        for j := i; j <= 6; j++ {

            b := false

            if i == j {

                b = true

            }

            d = append(d, Domino{i, j, b})

        }

    }

    return Dominoes{d, len(d)}
}

func kocokKartu(d *Dominoes) {

    rand.Seed(time.Now().UnixNano())

    for i := range d.kartu {

        j := rand.Intn(len(d.kartu))

        d.kartu[i], d.kartu[j] = d.kartu[j],
d.kartu[i]

    }

}

func ambilKartu(d *Dominoes) Domino {

    if d.sisa <= 0 {

        return Domino{-1, -1, false}

    }

}

```

```

    }

    k := d.kartu[d.sisa-1]

    d.sisa--

    return k
}

func gambarKartu(k Domino, suit int) int {
    if suit == 1 {
        return k.suit1
    }

    return k.suit2
}

func nilaiKartu(k Domino) int {
    return k.suit1 + k.suit2
}

func main() {
    deck := buatDomino()

    kocokKartu(&deck)

    fmt.Println("Ambil 5 kartu domino:\n")
}

```

```

        for i := 0; i < 5; i++ {

            k := ambilKartu(&deck)

            fmt.Printf("Kartu %d: [%d|%d] | nilai: %d\n",

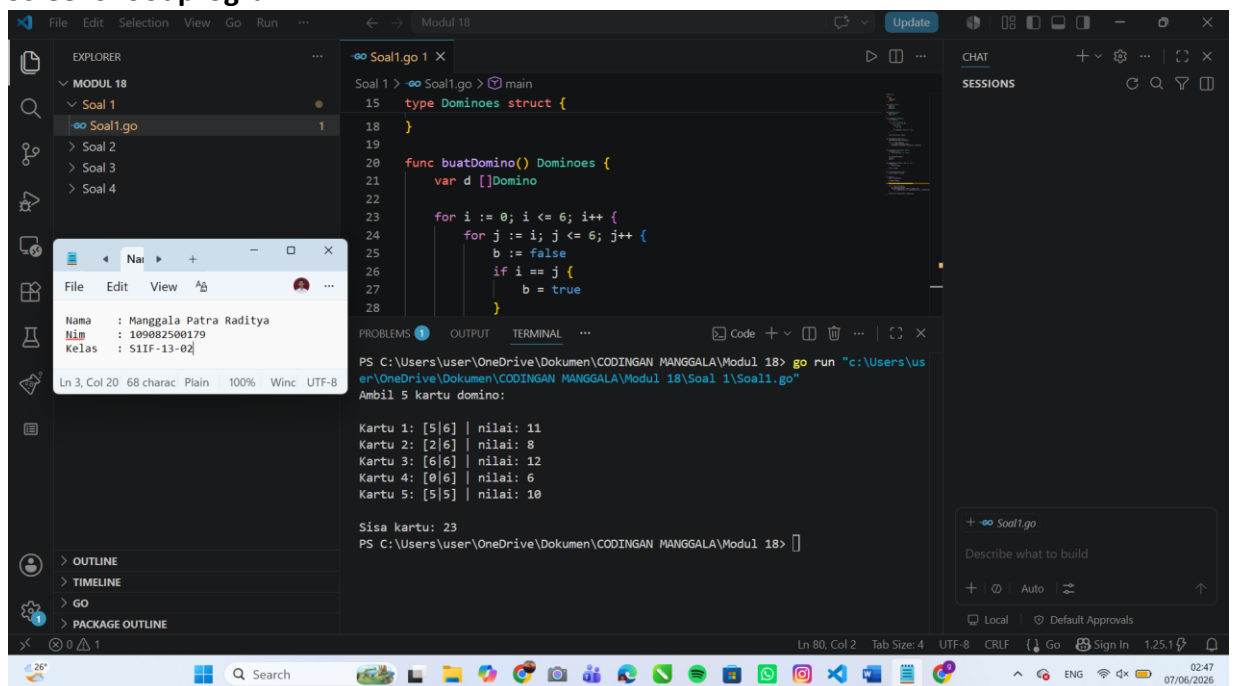
                i+1, gambarKartu(k, 1), gambarKartu(k,
2), nilaiKartu(k))

        }

        fmt.Println("\nSisa kartu:", deck.sisa)
    }
}

```

Screenshoot program



Deskripsi program

Program ini mengimplementasikan struktur data dan operasi dasar pada kartu domino dengan membentuk satu set berisi 28 kartu domino, masing-masing memiliki dua sisi bernilai 0 hingga 6 serta atribut penanda apakah kartu tersebut balak atau tidak. Program juga menyediakan proses pengacakan kartu (shuffle), pengambilan kartu dari dek, serta fungsi untuk menampilkan nilai setiap sisi kartu dan menghitung total nilai kedua sisi. Dengan mekanisme tersebut, program mensimulasikan pengelolaan deck domino secara sederhana seperti pada permainan nyata, termasuk pengambilan kartu secara acak dan pengurangan jumlah kartu yang tersisa dalam dek.

2. Soal 2

Source code

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

type Domino struct {
    suit1 int
    suit2 int
}

type Dominoes struct {
    kartu []Domino
    sisa  int
}

func buatDomino() Dominoes {
    var d []Domino

    for i := 0; i <= 6; i++ {
        for j := i; j <= 6; j++ {
```

```

        d = append(d, Domino{i, j})

    }

}

return Dominoes{d, len(d)}
}

func kocokKartu(d *Dominoes) {
    rand.Seed(time.Now().UnixNano())

    for i := range d.kartu {
        j := rand.Intn(len(d.kartu))

        d.kartu[i], d.kartu[j] = d.kartu[j],
d.kartu[i]
    }
}

func galiKartu(d *Dominoes, target Domino) Domino {
    for d.sisa > 0 {
        k := d.kartu[d.sisa-1]

        d.sisa--

        if k.suit1 == target.suit1 || k.suit1 ==
target.suit2 ||
            k.suit2 == target.suit1 || k.suit2 ==
target.suit2 {
            return k
        }
    }
}

```

```
    }

    }

    return Domino{-1, -1}
}

func sepasangKartu(a Domino, b Domino) bool {
    total := a.suit1 + a.suit2 + b.suit1 + b.suit2
    return total == 12
}

func main() {
    deck := buatDomino()
    kocokKartu(&deck)

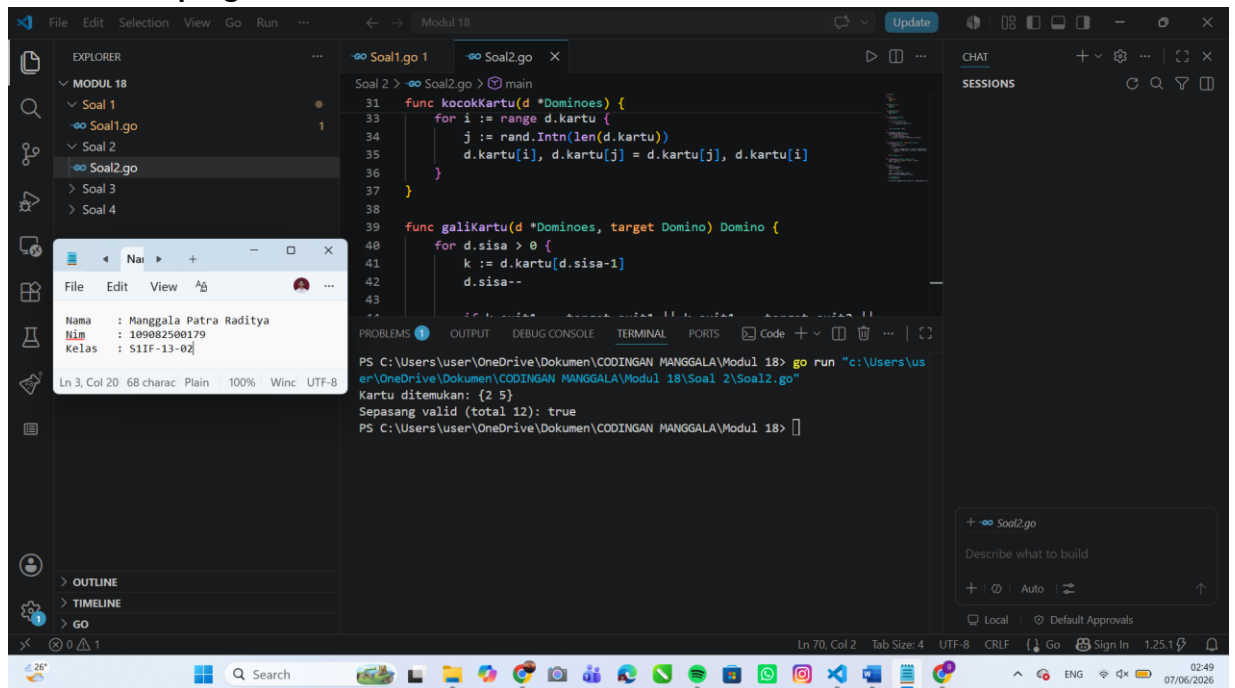
    target := Domino{3, 5}

    kartu := galiKartu(&deck, target)
    fmt.Println("Kartu ditemukan:", kartu)

    a := Domino{6, 3}
    b := Domino{2, 1}

    fmt.Println("Sepasang valid (total 12):",
        sepasangKartu(a, b))
}
```

Screenshoot program



```
Soal 2 > Soal2.go > main
31 func kocokKartu(d *Dominoes) {
32     for i := range d.kartu {
33         j := rand.Intn(len(d.kartu))
34         d.kartu[i], d.kartu[j] = d.kartu[j], d.kartu[i]
35     }
36 }
37
38
39 func galiKartu(d *Dominoes, target Domino) Domino {
40     for d.sisa > 0 {
41         k := d.kartu[d.sisa-1]
42         d.sisa--
43     }
44 }
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
PS C:\Users\User\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 18> go run "c:\Users\User\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 18\Soal 2\Soal2.go"
Kartu ditemukan: {2 5}
Sepasang valid (total 12): true
PS C:\Users\User\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 18>
```

Deskripsi program

Program ini mengimplementasikan operasi dasar pada struktur data kartu domino, termasuk pencarian kartu melalui prosedur galiKartu yang mengambil kartu dari tumpukan hingga ditemukan kartu yang memiliki kesamaan nilai gambar (suit) dengan kartu referensi yang diberikan. Selain itu, terdapat fungsi sepasangKartu yang digunakan untuk mengecek apakah dua kartu domino memenuhi kondisi tertentu, yaitu memiliki jumlah total nilai seluruh sisi kartu sebesar 12. Program juga dilengkapi dengan pembentukan deck domino dan proses pengacakan kartu untuk mensimulasikan kondisi permainan yang lebih realistis.

3. Soal 3

Source code

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

type Domino struct {
    suit1 int
    suit2 int
}

type Dominoes struct {
    kartu []Domino
    sisa  int
}

func buatDomino() Dominoes {
    var d []Domino

    for i := 0; i <= 6; i++ {
        for j := i; j <= 6; j++ {
```

```

        d = append(d, Domino{i, j})

    }

}

return Dominoes{d, len(d)}
}

func kocokKartu(d *Dominoes) {
    rand.Seed(time.Now().UnixNano())

    for i := range d.kartu {
        j := rand.Intn(len(d.kartu))

        d.kartu[i], d.kartu[j] = d.kartu[j],
d.kartu[i]
    }
}

func ambilKartu(d *Dominoes) Domino {
    if d.sisa <= 0 {
        return Domino{-1, -1}
    }

    k := d.kartu[d.sisa-1]

    d.sisa--

    return k
}

```

```
func gambarKartu(k Domino, sisi int) int {  
    if sisi == 1 {  
        return k.suit1  
    }  
    return k.suit2  
}  
  
func nilaiKartu(k Domino) int {  
    return k.suit1 + k.suit2  
}  
  
func main() {  
    deck := buatDomino()  
    kocokKartu(&deck)  
  
    fmt.Println("=== GAME GAPLEH SEDERHANA ===\n")  
  
    totalPlayer := 4  
    skor := make([]int, totalPlayer)  
  
    for p := 0; p < totalPlayer; p++ {  
        fmt.Printf("Player %d:\n", p+1)  
  
        for i := 0; i < 4; i++ {
```

```

        k := ambilKartu(&deck)

        fmt.Printf("    Kartu: [%d|%d] nilai=%d\n",
                    gambarKartu(k, 1), gambarKartu(k,
2), nilaiKartu(k))

        skor[p] += nilaiKartu(k)

    }

    fmt.Printf("    Total nilai Player %d = %d\n\n",
p+1, skor[p])

}

menang := 0

for i := 1; i < totalPlayer; i++ {

    if skor[i] > skor[menang] {

        menang = i

    }

}

    fmt.Println("Pemenang adalah Player", menang+1,
"dengan skor", skor[menang])

}

```

Screenshoot program

```
Soal3.go 1
48 func gambarKartu(k Domino, sisi int) int {
52     return k.suit2
53 }
54
55 func nilaiKartu(k Domino) int {
56     return k.suit1 + k.suit2
57 }
58
59 func main() {
60     deck := buatDomino()
61     kocokKartu(&deck)
62
63     // ... (code for Player 3 and Player 4) ...
64
65     Pemainang adalah Player 2 dengan skor 31
66 }

Player 3:
Kartu: [5]5 nilai=10
Kartu: [0]4 nilai=4
Kartu: [0]5 nilai=5
Kartu: [4]5 nilai=9
Total nilai Player 3 = 28

Player 4:
Kartu: [3]3 nilai=6
Kartu: [3]6 nilai=9
Kartu: [1]3 nilai=4
Kartu: [4]4 nilai=8
Total nilai Player 4 = 27
```

Deskripsi program

Program ini mengimplementasikan permainan domino sederhana berbasis konsep Gapleh, di mana setiap pemain mendapatkan sejumlah kartu dari tumpukan kartu domino yang telah diacak. Setiap kartu memiliki dua sisi dengan nilai 0 hingga 6, dan nilai kartu dihitung dari jumlah kedua sisinya. Pemain kemudian mengumpulkan total nilai dari kartu yang diperoleh, dan pemain dengan total nilai tertinggi dinyatakan sebagai pemenang. Program ini memanfaatkan operasi dasar mesin domino seperti mengocok kartu, mengambil kartu dari tumpukan, serta menghitung nilai kartu untuk mensimulasikan mekanisme permainan domino secara sederhana.

4. Soal 4

Source code

```
package main

import (
    "fmt"
)

var text string
var idx int

func start(input string) {
    text = input
    idx = 0
}

func maju() {
    if idx < len(text) {
        idx++
    }
}

func eop() bool {
    return idx >= len(text) || text[idx] == '.'
}
```

```
func cc() byte {  
    return text[idx]  
}  
  
func main() {  
    input := "LEARN TO PROGRAM WITH LEARNING LE. "  
  
    start(input)  
  
    totalChar := 0  
    countA := 0  
    countLE := 0  
  
    for !eop() {  
        c := cc()  
  
        totalChar++  
  
        if c == 'A' {  
            countA++  
        }  
  
        if idx < len(text)-1 {
```

```
        if text[idx] == 'L' && text[idx+1] == 'E'
    {

        countLE++

    }

    }

    maju()

}

var freqA float64

if totalChar > 0 {

    freqA = float64(countA) / float64(totalChar)

}

fmt.Println("Total karakter:", totalChar)

fmt.Println("Jumlah huruf A:", countA)

fmt.Println("Frekuensi A:", freqA)

fmt.Println("Jumlah kata LE:", countLE)

}
```

Screenshoot program

The screenshot shows a Go IDE with a project named 'MODUL 18'. The Explorer panel on the left lists files: 'Soal 1', 'Soal 2', 'Soal 3', 'Soal 3.go', and 'Soal 4'. A small window is open over the Explorer, displaying student information: 'Nama : Manggala Patra Raditya', 'Nim : 109082500179', and 'Kelas : 511F-13-02'. The main editor displays the code for 'Soal4.go', which defines a 'main' function. The code initializes counters for total characters, the letter 'A', and the letter 'L'. It then iterates through the input string 'A...LE...' (partially visible), incrementing the counters. The output panel shows the results: 'Total karakter: 33', 'Jumlah huruf A: 3', 'Frekuensi A: 0.09090909090909091', and 'Jumlah kata LE: 3'. The terminal panel shows the command 'go run "c:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 18\Soal 4\Soal4.go"' and the resulting output.

```
func main() {  
    totalChar := 0  
    countA := 0  
    countLE := 0  
  
    for loop() {  
        c := cc()  
  
        totalChar++  
  
        if c == 'A' {  
            countA++  
        }  
    }  
}
```

PS C:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 18> go run "c:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 18\Soal 4\Soal4.go"
Total karakter: 33
Jumlah huruf A: 3
Frekuensi A: 0.09090909090909091
Jumlah kata LE: 3
PS C:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 18>

Deskripsi program

Program ini mengimplementasikan mesin abstrak karakter yang membaca sebuah untaian karakter hingga ditemukan penanda akhir berupa titik ('.'). Mesin menyediakan operasi dasar seperti start untuk inisialisasi, maju untuk berpindah ke karakter berikutnya, eop untuk mendeteksi akhir input, dan cc untuk mengambil karakter yang sedang dibaca. Selama proses pembacaan, program menghitung jumlah seluruh karakter, jumlah kemunculan huruf 'A', frekuensi huruf 'A' terhadap total karakter, serta mendeteksi pola pasangan huruf berturutan "LE" dalam teks. Dengan pendekatan ini, program mensimulasikan cara kerja mesin karakter sederhana untuk analisis string secara sekuensial.